

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN CUỐI KÌ
Đề tài số 14: Thiết kế Game lật thẻ (Memory Game)
trên giả lập RARS

Môn: Thực Hành Kiến Trúc Máy Tính

Mã lớp: 166228

Mã học phần: IT3280

Giảng viên hướng dẫn: TS. Phạm Ngọc Hưng

Họ và tên sinh viên: Nguyễn Thị Thanh Trúc

MSSV: 202417052

Hà Nội, tháng 06 năm 2026

Mục lục

I. GIỚI THIỆU ĐỀ TÀI & THÔNG SỐ CẤU HÌNH	3
1. Mô tả đề tài.....	3
2. Cấu hình các công cụ giả lập trên RARS.....	4
II. THIẾT KẾ DỮ LIỆU VÀ PHÂN BỐ BỘ NHỚ	5
III. CHI TIẾT THIẾT KẾ CÁC CHỨC NĂNG CỐT LÕI.....	8
1. Menu điều khiển và xác thực dữ liệu đầu vào.....	8
2. Khôi phục trạng thái hệ thống (reset_game_state).....	11
3. Vòng lặp trò chơi, quản lý thời gian và nhập liệu bất tuần tự.....	12
4. Logic xử lý Lật thẻ và so khớp cặp màu	13
5. Thiết kế chương trình đồ họa cấp thấp (Vẽ hình chữ nhật)	16
IV. DEMO CHƯƠNG TRÌNH & HƯỚNG DẪN VẬN HÀNH	18
1. Quy trình chạy chương trình	18
2. Các kịch bản thử nghiệm và kết quả thực tế.....	18
V. KẾT LUẬN & HƯỚNG PHÁT TRIỂN	23
1. Kết quả đạt được của Bài tập lớn.....	23
2. Hạn chế và Hướng phát triển của đề tài.....	23

I. GIỚI THIỆU ĐỀ TÀI & THÔNG SỐ CẤU HÌNH

1. Mô tả đề tài

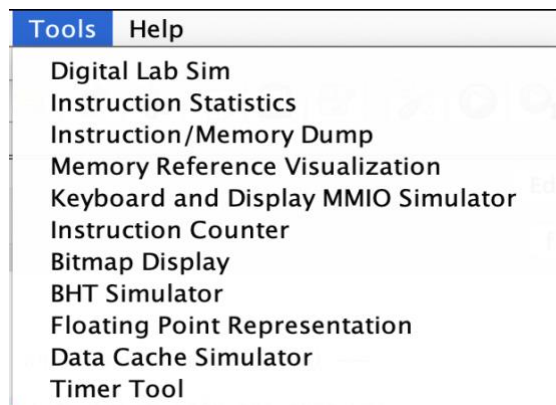
Bài tập lớn tập trung nghiên cứu và hiện thực hóa đề tài số **14: Game lật thẻ (Memory Game)** trên nền tảng kiến trúc RISC-V. Trò chơi được thiết kế trên một bàn cờ kích thước 4×4 chứa 16 ô tương ứng với 8 cặp màu sắc khác nhau được sinh ngẫu nhiên.

- **Tính ngẫu nhiên (Shuffle):** Khi bắt đầu trò chơi, chương trình sử dụng thuật toán hoán vị Fisher-Yates kết hợp dịch vụ hệ thống Syscall 42 (Sinh số ngẫu nhiên có giới hạn) để xáo trộn vị trí của 8 cặp màu trong mảng cards, đảm bảo tính độc lập và ngẫu nhiên giữa các lượt chơi.
- **Cơ chế điều khiển mô phỏng bàn phím ma trận (KeyMatrix Simulation):** Để đáp ứng yêu cầu sử dụng bàn phím ma trận (4×4 KeyMatrix) trên phần cứng thực tế, chương trình đã phần mềm hóa cơ chế này bằng cách sử dụng bộ mô phỏng bàn phím (Keyboard Simulator) qua MMIO. Người chơi sử dụng các phím W (Lên), S (Xuống), A (Trái), D (Phải) để thay đổi tọa độ hàng/cột (giống như thao tác quét hàng/cột trên KeyMatrix) làm di chuyển khung viền con trỏ màu trắng (color_cur), và nhấn phím SPACE để kích hoạt lật thẻ tại giao điểm tọa độ đang chọn.
- **Logic so khớp (Matching Logic):**
 - Hệ thống cho phép lật tạm thời tối đa 2 thẻ trong một lượt.
 - Nếu hai thẻ **khớp màu**, trạng thái của chúng được cập nhật thành đã mở (trạng thái 1).
 - Nếu hai thẻ **khác màu**, chương trình sử dụng dịch vụ trì hoãn Syscall 32 (Sleep) trong 800 ms để người chơi ghi nhớ vị trí, sau đó tự động úp cả hai thẻ về trạng thái ban đầu (trạng thái 0).
- **Hạn định thời gian (Time Limit):** Trò chơi giới hạn thời gian hoàn thành là **90 giây (90.000 ms)**. Chương trình liên tục truy vấn thời gian thực tế của CPU thông qua dịch vụ Syscall 30 (Get Time) để tính toán hiệu số thời gian đã trôi qua và đưa ra quyết định dừng cuộc chơi khi quá hạn (chuyển sang trạng thái Lose).
- **Khả năng tái khởi động (Replayability):** Trò chơi tương tác với người dùng qua màn hình Console (Run I/O) bằng một Menu điều khiển có kiểm tra tính hợp lệ dữ liệu đầu vào. Sau khi có kết quả Thắng/Thua, hệ thống tự

động dọn dẹp bộ nhớ đồ họa và đưa người chơi quay lại Menu chính mà không cần khởi động lại giả lập RARS.

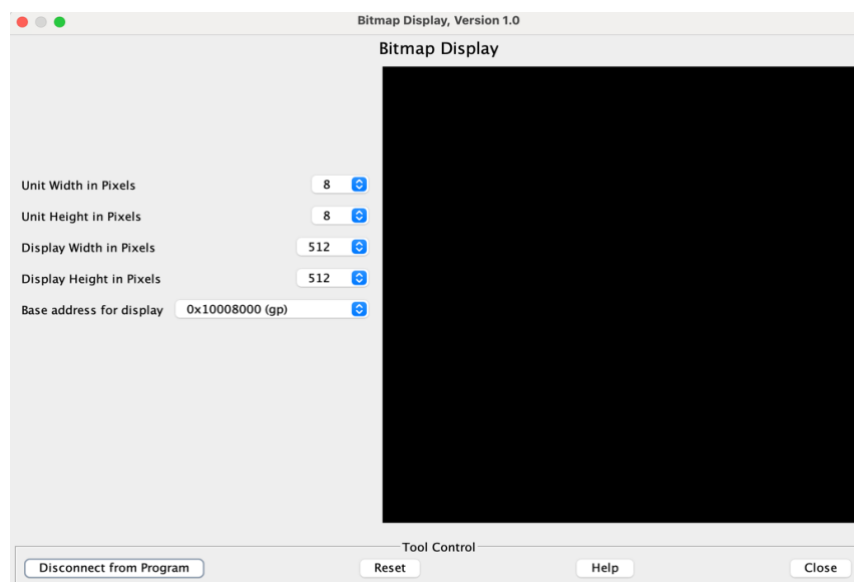
2. Cấu hình các công cụ giả lập trên RARS

Để chương trình hoạt động chính xác, các công cụ ngoại vi cần được thiết lập theo các thông số kỹ thuật sau:



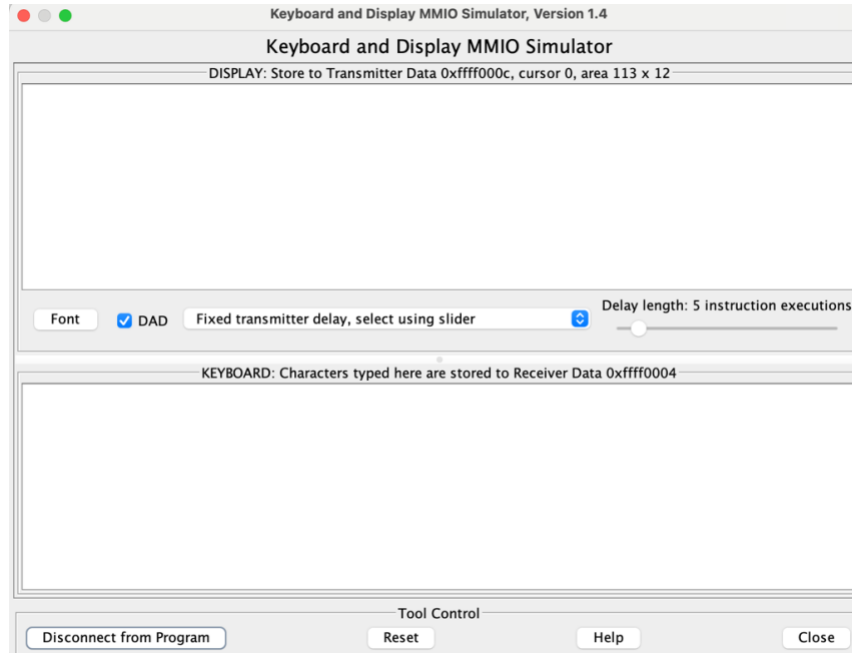
- **Bitmap Display**

- *Unit Width in Pixels: 8*
- *Unit Height in Pixels: 8*
- *Display Width in Pixels: 512*
- *Display Height in Pixels: 512*
- *Base Address: 0x10008000 (gp)*



- **Keyboard and Display Simulator:**

- Sử dụng cơ chế MMIO (Memory-Mapped I/O) với địa chỉ điều khiển bàn phím là 0xffff0000 và địa chỉ dữ liệu bàn phím là 0xffff0004.



II. THIẾT KẾ DỮ LIỆU VÀ PHÂN BỐ BỘ NHỚ

Bộ nhớ dữ liệu (.data) và các hằng số (.eqv) được tổ chức chặt chẽ nhằm tối ưu hóa dung lượng bộ nhớ, bảo toàn tính toàn vẹn dữ liệu và tăng tốc độ truy xuất của CPU RISC-V.

**** Code + giải thích:**

```
# --- HẰNG SỐ ẢNH XẠ BỘ NHỚ (MEMORY-MAPPED I/O CONSTANTS) ---
.eqv DISPLAY_BASE 0x10008000
.eqv KBD_CTRL    0xffff0000
.eqv KBD_DATA    0xffff0004

.data
# Mảng lưu giá trị của 16 thẻ (8 cặp màu từ 0 đến 7)
cards:    .byte 0, 0, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7
```

```

# Trạng thái của từng thẻ: 0 = Úp, 1 = Đã mở (khớp), 2 = Đang tạm lật
states:      .byte 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

# Bảng mã màu RGB cho 8 cặp hình ảnh/thẻ
colors:      .word 0xFF0000, 0x00FF00, 0x0000FF, 0xFFFF00, 0xFF00FF,
0x00FFFF, 0xFF8000, 0x800080
color_back:  .word 0x222222 # Màu nền màn hình (Xám tối)
color_down:  .word 0x4A6984 # Màu khi thẻ đang úp
color_cur:   .word 0xFFFFFF # Màu viền con trỏ chọn (Trắng)

# Các biến trạng thái trò chơi
cursor_x:    .word 0        # Vị trí X của con trỏ (0 -> 3)
cursor_y:    .word 0        # Vị trí Y của con trỏ (0 -> 3)
flipped_cnt: .word 0        # Số lượng thẻ đang lật tạm thời trong lượt (0, 1, hoặc 2)
first_idx:   .word -1       # Chỉ số của thẻ lật đầu tiên
matches:     .word 0        # Số cặp đã hoàn thành (nếu đạt 8 thì thắng)

# Biến thời gian
start_time:  .word 0        # Lưu thời điểm bắt đầu chơi (ms)

```

**** Phân tích giải pháp thiết kế bộ nhớ:**

1. Phân tách bộ nhớ động và bộ nhớ tĩnh

- cards (Tĩnh): Lưu trữ cố định giá trị định danh màu sắc (từ 0 đến 7, mỗi giá trị xuất hiện đúng 2 lần) của 16 thẻ. Mảng này chỉ bị xáo trộn vị trí phần tử bởi thuật toán ngẫu nhiên lúc bắt đầu game và giữ nguyên giá trị trong suốt lượt chơi.
- states (Động): Lưu trữ trạng thái vật lý của thẻ tại thời điểm hiện tại:
 - 0: Thẻ đang úp.
 - 1: Thẻ đã được mở (khớp thành công).
 - 2: Thẻ đang lật tạm thời (chờ so khớp với thẻ thứ hai).
- *Ý nghĩa giải pháp:* Việc tách biệt hoàn toàn giữa **giá trị nội dung thẻ** (cards) và **trạng thái hiển thị** (states) là một giải pháp thiết kế hướng đối tượng kinh điển được áp dụng vào Hợp ngữ. Điều này giúp hệ thống quản lý logic so khớp cực kỳ đơn giản: Khi hai ô không khớp, chương trình

chỉ cần ghi đè giá trị 0 vào mảng states tại các chỉ số tương ứng mà không làm ảnh hưởng đến mảng màu gốc cards.

2. Tối ưu hóa dung lượng lưu trữ (Kiểu dữ liệu Byte vs Word)

- **Sử dụng .byte (1 byte = 8 bits):** Hai mảng cards và states chỉ chứa các giá trị số nguyên rất nhỏ (từ 0 đến 7). Do đó, việc khai báo chúng dưới kiểu dữ liệu .byte giúp tiết kiệm đáng kể bộ nhớ. 16 thẻ chỉ tiêu tốn đúng 16 bytes bộ nhớ, thay vì phải mất tới 64 bytes nếu khai báo kiểu .word (4 bytes).
- **Sử dụng .word (4 bytes = 32 bits):** Bảng màu colors, color_back, color_down, color_cur bắt buộc phải sử dụng kiểu dữ liệu .word vì mã màu RGB của hệ thống là mã màu 24-bit (ví dụ: 0xFF0000). Kiểu .word cũng được áp dụng cho các biến điều khiển trò chơi (cursor_x, cursor_y, flipped_cnt, first_idx, matches, start_time) để đảm bảo các chỉ thị nạp từ bộ nhớ như lw (load word) và lưu vào bộ nhớ sw (store word) hoạt động đồng bộ với các thanh ghi 32-bit của kiến trúc RISC-V mà không cần thực hiện các bước chuyển đổi dữ liệu trung gian.

3. Cơ chế giao tiếp ngoại vi qua Ánh xạ bộ nhớ (MMIO)

Chương trình không sử dụng các lệnh xuất/nhập hệ thống thông thường để điều khiển thiết bị ngoại vi, mà tận dụng cơ chế **Memory-Mapped I/O (MMIO)**:

- **DISPLAY_BASE (0x10008000):** Nằm ở vùng nhớ Heap. Mọi thao tác ghi dữ liệu màu sắc (lệnh sw) vào vùng nhớ này sẽ lập tức được phần cứng ánh xạ thành các pixel màu hiển thị trên màn hình Bitmap Display.
- **KBD_CTRL (0xffff0000) và KBD_DATA (0xffff0004):** Nằm ở vùng nhớ hệ thống dành riêng cho thiết bị ngoại vi. Việc đọc các địa chỉ này giúp CPU kiểm tra trạng thái hoạt động và nhận dữ liệu quét từ bộ mô phỏng bàn phím theo thời gian thực mà không làm nghẽn luồng xử lý chính.

III. CHI TIẾT THIẾT KẾ CÁC CHỨC NĂNG CỐT LÕI

Dưới đây là các đoạn mã nguồn chính được chia nhỏ theo chức năng để phân tích logic xử lý.

1. Menu điều khiển và xác thực dữ liệu đầu vào

Để đảm bảo chương trình vận hành liên tục mà không bị gián đoạn khi người dùng nhập sai, hệ thống triển khai một Menu tương tác trực quan qua Console (Run I/O) kết hợp với cơ chế kiểm tra lỗi nghiêm ngặt.

**** Code + giải thích:**

`show_menu:`

`# 1. Hiện thị Menu chính ra tab Run I/O`

`la a0, menu_msg`

`jal print_string`

`# 2. Đọc lựa chọn đầu vào của người dùng (Xác thực tính hợp lệ)`

`li a7, 5 # Syscall 5: Read Integer`

`ecall # Trả về số nguyên ở a0`

`li t0, 1`

`beq a0, t0, play_option`

`li t0, 2`

`beq a0, t0, rules_option`

`li t0, 3`

`beq a0, t0, exit_option`

`# Trường hợp nhập sai giá trị ngoài khoảng [1, 3]`

`la a0, invalid_msg`

`jal print_string`

`j show_menu`


```

# --- CHI TIẾT CÁC NHÃN ĐIỀU HƯỚNG CHỨC NĂNG ---
rules_option:
    # Hiển thị hướng dẫn rồi quay lại Menu
    jal show_rules
    j show_menu

exit_option:
    # Thoát chương trình
    la a0, exit_msg
    jal print_string
    li a7, 10          # Syscall 10: Exit
    ecall

play_option:
    # 3. Reset toàn bộ trạng thái game về ban đầu (Phục vụ chơi lại nhiều lần)
    jal reset_game_state

    # Hiển thị thông báo bắt đầu
    la a0, intro_msg
    jal print_string

    # Trộn thẻ ngẫu nhiên
    jal shuffle_cards

    # Ghi lại thời điểm bắt đầu trò chơi (milliseconds)
    li a7, 30          # Syscall 30: Get Time
    ecall
    la t0, start_time
    sw a0, 0(t0)

    # Vẽ màn hình ban đầu
    jal draw_screen

```

**** Phân tích giải pháp kỹ thuật:**

- **Cơ chế xác thực và điều hướng rẽ nhánh:** Chương trình sử dụng chỉ thị so sánh và rẽ nhánh beq để đối chiếu giá trị lựa chọn của người dùng trong thanh ghi a0 với các hằng số đích lưu tại t0. Khi phát hiện giá trị trùng khớp, hệ thống thực hiện rẽ nhánh điều khiển luồng xử lý tương ứng với từng chức năng riêng biệt:
 - **Lựa chọn số 1 (play_option - Bắt đầu chơi):** Khi người dùng nhập 1, chương trình sẽ gọi chương trình con reset_game_state để dọn dẹp toàn bộ dữ liệu trạng thái cũ, in lời chào bắt đầu qua intro_msg, xáo trộn vị trí các thẻ bài ngẫu nhiên qua hàm shuffle_cards, ghi lại mốc thời gian hệ thống bằng Syscall 30 lưu vào biến start_time, vẽ giao diện bàn cờ ban đầu qua hàm draw_screen và chuyển tiếp sang vòng lặp trò chơi.
 - **Lựa chọn số 2 (rules_option - Xem hướng dẫn):** Khi người dùng nhập 2, chương trình chuyển hướng thực thi tới nhãn rules_option, gọi chương trình con show_rules để xuất toàn bộ thông tin hướng dẫn điều khiển bàn phím mô phỏng KeyMatrix (W, A, S, D, Space) ra màn hình Console. Sau khi in xong, hệ thống tự động nhảy về nhãn show_menu để hiển thị lại Menu chính.
 - **Lựa chọn số 3 (exit_option - Thoát game):** Khi người dùng nhập 3, hệ thống nhảy đến nhãn exit_option để xuất lời chào tạm biệt (exit_msg) và gọi lệnh hệ thống Syscall 10 (Exit) nhằm chấm dứt chương trình một cách an toàn.
- **Xử lý ngoại lệ (Exception Handling):** Nếu người dùng nhập một giá trị nằm ngoài tập hợp số nguyên {1, 2, 3}, luồng thực thi của CPU sẽ bỏ qua các lệnh rẽ nhánh và rơi xuống đoạn mã xử lý lỗi. Chương trình sẽ xuất chuỗi thông báo lỗi invalid_msg ra Console, sau đó thực hiện lệnh nhảy không điều kiện j show_menu để buộc người dùng nhập lại từ đầu mà không làm crash hay dừng hệ thống đột ngột.

2. Khôi phục trạng thái hệ thống (reset_game_state)

Chức năng này dọn dẹp các biến nhớ trung gian và khôi phục trạng thái ban đầu của trò chơi, cho phép người dùng chơi lại nhiều lần mà không cần khởi động lại trình giả lập RARS.

**** Code + giải thích:**

```
# --- Chương trình con dọn dẹp và khôi phục trạng thái ban đầu ---
reset_game_state:
    # 1. Reset mảng trạng thái thẻ (states) 16 phần tử về 0 (úp thẻ)
    la t0, states
    li t1, 0          # index i
    li t2, 16         # size
reset_states_loop:
    bge t1, t2, reset_states_done
    add t3, t0, t1
    sb zero, 0(t3)
    addi t1, t1, 1
    j reset_states_loop
reset_states_done:

    # 2. Đặt lại các biến trạng thái trò chơi về mặc định
    la t0, cursor_x
    sw zero, 0(t0)
    la t0, cursor_y
    sw zero, 0(t0)
    la t0, flipped_cnt
    sw zero, 0(t0)
    la t0, first_idx
    li t1, -1
    sw t1, 0(t0)
    la t0, matches
    sw zero, 0(t0)
    ret
```

**** Phân tích giải pháp kỹ thuật:**

- **Tối ưu hóa ghi bộ nhớ:** Vòng lặp `reset_states_loop` sử dụng lệnh `sb` (store byte) để truy cập chính xác từng byte trong bộ nhớ. Việc này đảm bảo tính toàn vẹn của các ô nhớ liền kề trong mảng `states`.
- **Khôi phục trạng thái logic:** Các biến điều khiển hệ thống (`cursor_x`, `cursor_y`, `flipped_cnt`, `matches`) được thiết lập lại về 0 bằng chỉ thị ghi từ `sw` (store word) kết hợp với thanh ghi cứng `zero`. Biến `first_idx` được đưa về -1 để đánh dấu hệ thống chưa ghi nhận bất kỳ thẻ nào được lật trong lượt chơi mới.

3. Vòng lặp trò chơi, quản lý thời gian và nhập liệu bất tuần tự

Vòng lặp chính (`game_loop`) được thiết kế theo mô hình thời gian thực để giải quyết đồng thời hai nhiệm vụ: theo dõi giới hạn thời gian 90 giây và xử lý tín hiệu bấm phím từ người dùng một cách trực tiếp mà không gây dừng luồng CPU.

**** Code + giải thích:**

```
game_loop:
# 4. Kiểm tra giới hạn thời gian (Time Limit Check: 90s)
li a7, 30          # Lấy thời gian hiện tại
ecall
la t0, start_time
lw t1, 0(t0)
sub t2, a0, t1     # t2 = thời gian đã trôi qua (ms)
li t3, 90000       # Giới hạn 90 giây = 90.000 ms
bge t2, t3, lose_state # Nếu đã quá 90s, chuyển sang trạng thái thua

# 5. Đọc phím bấm từ MMIO Keyboard (Không chặn / Non-blocking)
li t0, KBD_CTRL
lw t1, 0(t0)       # Đọc thanh ghi điều khiển bàn phím
andi t1, t1, 1     # Kiểm tra bit sẵn sàng (Ready bit)
beq t1, zero, game_loop # Nếu chưa có phím bấm, tiếp tục vòng lặp

# Nếu có phím, thực hiện đọc dữ liệu phím từ MMIO
li t0, KBD_DATA
lw s0, 0(t0)       # Lưu ký tự đọc được vào s0
```

```

# Xử lý lọc phím hợp lệ và điều hướng hành động tương ứng
li t0, 119      # Phím 'w'
beq s0, t0, move_up
li t0, 115      # Phím 's'
beq s0, t0, move_down
li t0, 97       # Phím 'a'
beq s0, t0, move_left
li t0, 100      # Phím 'd'
beq s0, t0, move_right
li t0, 32       # Phím 'Space'
beq s0, t0, action_flip
j game_loop     # Phím không hợp lệ sẽ bị bỏ qua

```

** Phân tích giải pháp kỹ thuật:

- **Cơ chế Polling:** Thay vì sử dụng ngắt hệ thống phức tạp, chương trình liên tục thực hiện truy vấn trạng thái hoạt động của thanh ghi điều khiển ngoại vi KBD_CTRL tại địa chỉ 0xffff0000. Chỉ khi người dùng bấm phím làm kích hoạt bit Ready (bit 0) lên 1, chương trình mới tiến hành đọc mã ASCII tại thanh ghi dữ liệu KBD_DATA (0xffff0004).
- **Đồng bộ thời gian thực:** Nhờ cơ chế Polling này, chu kỳ kiểm tra thời gian trôi qua diễn ra liên tục sau mỗi vài micro giây, đảm bảo bộ đếm thời gian hoạt động chính xác theo thời gian thực và tự động ngắt game ngay lập tức khi chạm mốc 90 giây.

4. Logic xử lý Lật thẻ và so khớp cặp màu

Module này quản lý máy trạng thái (State Machine) của trò chơi khi người dùng chọn lật một thẻ bất kỳ bằng phím Space.

** Code + giải thích:

```

action_flip:
# Tính toán chỉ số mảng 1D từ tọa độ con trỏ (row * 4 + col)
la t0, cursor_x
lw t1, 0(t0)      # t1 = col
la t2, cursor_y

```

```

lw  t3, 0(t2)      # t3 = row
slli t4, t3, 2      # row * 4
add  s1, t4, t1     # s1 = idx (0 -> 15)

# Kiểm tra trạng thái thẻ tại vị trí này
la  t0, states
add  t0, t0, s1
lb  t1, 0(t0)      # Đọc trạng thái thẻ hiện tại
bne  t1, zero, game_loop # Nếu thẻ đã mở hoặc lật rồi thì bỏ qua

# Xử lý lật thẻ dựa trên biến flipped_cnt
la  t2, flipped_cnt
lw  t3, 0(t2)

bne  t3, zero, handle_second_card

# --- Trường hợp đây là thẻ lật đầu tiên trong lượt ---
li  t4, 2
sb  t4, 0(t0)      # Đặt trạng thái thẻ này thành 2 (đang tạm lật)
la  t5, first_idx
sw  s1, 0(t5)      # Ghi nhận chỉ số của thẻ đầu tiên
li  t3, 1
sw  t3, 0(t2)      # Tăng flipped_cnt lên 1
j   update_and_loop

handle_second_card:
li  t5, 1
bne  t3, t5, game_loop # Chỉ xử lý nếu flipped_cnt hiện tại bằng 1

# Đặt thẻ thứ 2 thành trạng thái lật tạm thời
li  t4, 2
sb  t4, 0(t0)

# Vẽ lại màn hình ngay để hiển thị cả 2 thẻ vừa lật
jal  draw_screen

# Lấy thông tin thẻ thứ nhất

```

```

la t5, first_idx
lw s2, 0(t5)      # s2 = idx của thẻ đầu tiên

# So sánh giá trị (màu sắc) của 2 thẻ
la t0, cards
add t1, t0, s1
lb t2, 0(t1)      # t2 = giá trị thẻ thứ hai
add t3, t0, s2
lb t4, 0(t3)      # t4 = giá trị thẻ thứ nhất

beq t2, t4, cards_match

# --- Trường hợp 2 thẻ KHÔNG KHỚP màu ---
li a0, 800
li a7, 32          # Syscall Sleep 800ms để người chơi nhìn kịp
ecall

# Đặt cả 2 thẻ quay lại trạng thái úp (0)
la t0, states
add t1, t0, s1
sb zero, 0(t1)
add t2, t0, s2
sb zero, 0(t2)
j reset_turn

cards_match:
# --- Trường hợp 2 thẻ KHỚP màu ---
la t0, states
add t1, t0, s1
li t3, 1
sb t3, 0(t1)      # Đặt trạng thái thẻ 2 thành đã khớp (1)
add t2, t0, s2
sb t3, 0(t2)      # Đặt trạng thái thẻ 1 thành đã khớp (1)

# Tăng số lượng cặp đã khớp thành công
la t4, matches
lw t5, 0(t4)

```

```
addi t5, t5, 1
sw t5, 0(t4)
```

reset_turn:

```
# Reset lượt chơi hiện tại
la t2, flipped_cnt
sw zero, 0(t2)
la t5, first_idx
li t6, -1
sw t6, 0(t5)
j update_and_loop
```

** Phân tích giải pháp kỹ thuật:

- **Trạng thái trung gian (State 2):** Trạng thái 2 (đang tạm lậ) là yếu tố quyết định để hiển thị màu sắc của thẻ lên màn hình trước khi CPU thực hiện so sánh logic.
- **Trì hoãn bằng phần cứng (Hardware Sleep):** Nếu hai thẻ không khớp, lệnh Syscall 32 với tham số a0 = 800 tạm dừng luồng xử lý của trò chơi trong đúng 800 mili giây. Điều này giữ nguyên hình ảnh hai thẻ đang mở trên màn hình Bitmap Display trước khi CPU thực thi lệnh ghi đè giá trị 0(úp thẻ), giúp nâng cao trải nghiệm ghi nhớ của người chơi.

5. Thiết kế chương trình đồ họa cấp thấp (Vẽ hình chữ nhật)

Chương trình con draw_rect chịu trách nhiệm render các ô cờ và hình nền trực tiếp lên vùng nhớ ánh xạ của Bitmap Display.

** Code + giải thích:

```
# --- Hàm vẽ hình chữ nhật đặc (Rectangle) ---
# Tham số đầu vào: a0 = x, a1 = y, a2 = w, a3 = h, a4 = color
draw_rect:
    li t0, DISPLAY_BASE
    mv t1, a1          # t1 = cy (current y)
    add t2, a1, a3      # t2 = y + h
```



```

outer_loop:
    bge t1, t2, rect_done
    mv  t3, a0          # t3 = cx (current x)
    add t4, a0, a2      # t4 = x + w

    # Tính toán địa chỉ nền cho dòng hiện tại: base + (cy * 64 * 4) -> cy * 256
    slli t5, t1, 8
    add t5, t5, t0

inner_loop:
    bge t3, t4, inner_done
    slli t6, t3, 2      # cx * 4
    add t6, t5, t6      # Địa chỉ pixel đích
    sw  a4, 0(t6)
    addi t3, t3, 1
    j   inner_loop
inner_done:
    addi t1, t1, 1
    j   outer_loop
rect_done:
    ret

```

** Phân tích giải pháp kỹ thuật:

- **Công thức ánh xạ tọa độ tuyến tính:** Bộ đệm khung đồ họa của hệ thống được tổ chức dưới dạng mảng một chiều liên tục trong bộ nhớ. Để vẽ một điểm tại tọa độ hai chiều (x, y), chương trình thực hiện tính toán địa chỉ vật lý theo công thức:

$$\text{Address} = \text{DISPLAY_BASE} + (y \times \text{Dòng}) + (x \times \text{Pixel})$$

Với độ phân giải logic là 64×64 ô và mỗi ô màu chiếm 4 bytes (.word), độ dài bộ nhớ của một dòng là $64 \times 4 = 256$ bytes. Công thức tính toán cụ thể được ánh xạ là:

$$\text{Address} = \text{DISPLAY_BASE} + (y \times 256) + (x \times 4)$$

- **Tối ưu hóa hiệu năng bằng phép dịch bit (Bit-shifting):** Thay vì sử dụng chỉ thị nhân mul của CPU (tốn nhiều chu kỳ máy), chương trình tối ưu hóa bằng cách sử dụng các chỉ thị dịch bit trái logic:
 - Phép toán $y \times 256$ được thực hiện bằng slli t5, t1, 8 ($2^8 = 256$).
 - Phép toán $x \times 4$ được thực hiện bằng slli t6, t3, 2 ($2^2 = 4$).
 Giải pháp dịch bit này giúp chương trình con vẽ đồ họa thực thi cực nhanh, giảm thiểu tối đa hiện tượng nhấp nháy màn hình (flickering) khi cập nhật hình ảnh bàn cờ theo thời gian thực.

IV. DEMO CHƯƠNG TRÌNH & HƯỚNG DẪN VẬN HÀNH

1. Quy trình chạy chương trình

1. Mở trình giả lập **RARS**.
2. Nạp tệp mã nguồn .asm.
3. Nhấn phím F3 để biên dịch mã nguồn.
4. Mở menu **Tools** -> Chọn **Bitmap Display** và thiết lập các thông số cấu hình như mục I.2, sau đó nhấn **Connect to Program**.
5. Mở menu **Tools** -> Chọn **Keyboard and Display Simulator**, sau đó nhấn **Connect to Program**.
6. Nhấn phím F5 (Run) để bắt đầu thực thi chương trình.

2. Các kịch bản thử nghiệm và kết quả thực tế

- **Kịch bản 1: Giao diện Menu chính trên Console**
 - *Mô tả:* Khi vừa chạy chương trình, màn hình Run I/O sẽ hiển thị Menu yêu cầu nhập lựa chọn. Nếu nhập số nằm ngoài khoảng 1-3, thông báo lỗi sẽ xuất hiện.
 - TH1: Nhập 1

```
===== MENU CHINH =====
1. Bat dau choi game (Play)
2. Xem huong dan choi (Rules)
3. Thoat chuong trinh (Exit)
Nhap lua chon cua ban (1-3): 1
```

```
=====
BAT DAU TRO CHOI! Ban co toi da 90 giay.
Hay click vao o Keyboard Simulator de di chuyen va lat the!
=====
```

○ Th2: Nhập 2

```
===== MENU CHINH =====
1. Bat dau choi game (Play)
2. Xem huong dan choi (Rules)
3. Thoat chuong trinh (Exit)
Nhap lua chon cua ban (1-3): 2

===== HUONG DAN CHOI =====
- Di chuyen con tro chon bang cac phim: W (Len), S (Xuong), A (Trai), D (Phai).
- Nhan phim SPACE (Khoang trang) de lat o dang chon.
- Luat choi: Tim ra tat ca 8 cap o cung mau trong vong 90 giay.
- Tro choi cho phep choi lai nhieu lan thong qua Menu.

===== MENU CHINH =====
1. Bat dau choi game (Play)
2. Xem huong dan choi (Rules)
3. Thoat chuong trinh (Exit)
Nhap lua chon cua ban (1-3): 1
```

○ TH3: Nhập 3

```
===== MENU CHINH =====
1. Bat dau choi game (Play)
2. Xem huong dan choi (Rules)
3. Thoat chuong trinh (Exit)
Nhap lua chon cua ban (1-3): 3

Chuong trinh ket thuc. Cam on ban da trai nghiem!

-- program is finished running (0) --
```

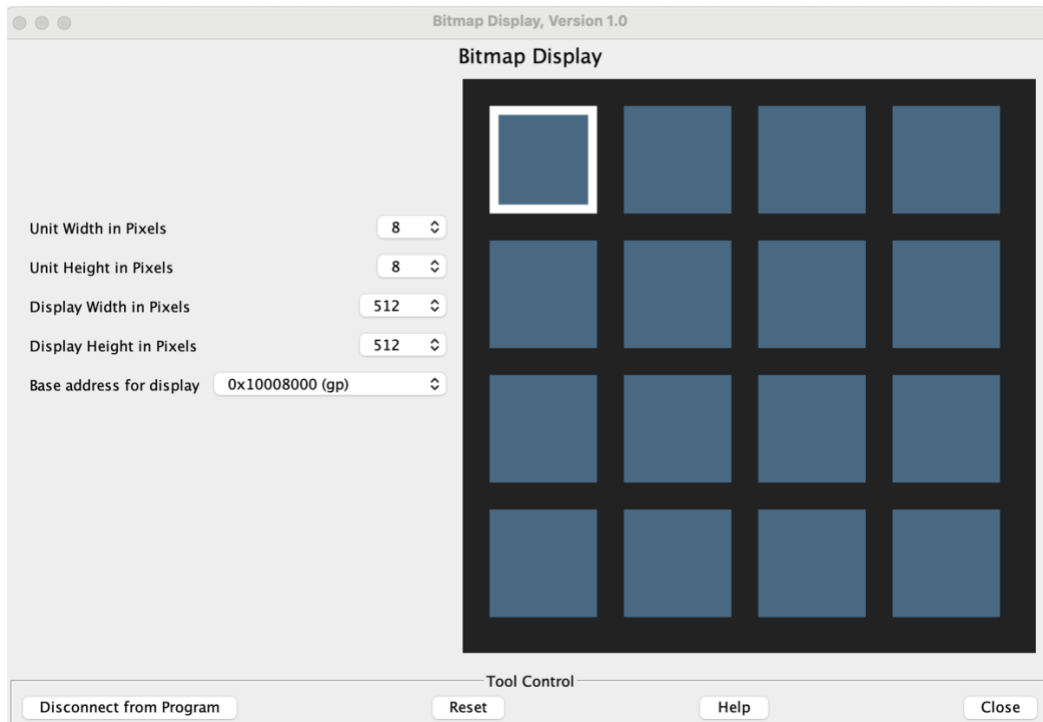
○ TH4: Nhập số khác

```
===== MENU CHINH =====
1. Bat dau choi game (Play)
2. Xem huong dan choi (Rules)
3. Thoat chuong trinh (Exit)
Nhap lua chon cua ban (1-3): 4
-> Lua chon khong hop le! Vui long chi nhap so 1, 2 hoac 3.

===== MENU CHINH =====
1. Bat dau choi game (Play)
2. Xem huong dan choi (Rules)
3. Thoat chuong trinh (Exit)
Nhap lua chon cua ban (1-3):
```

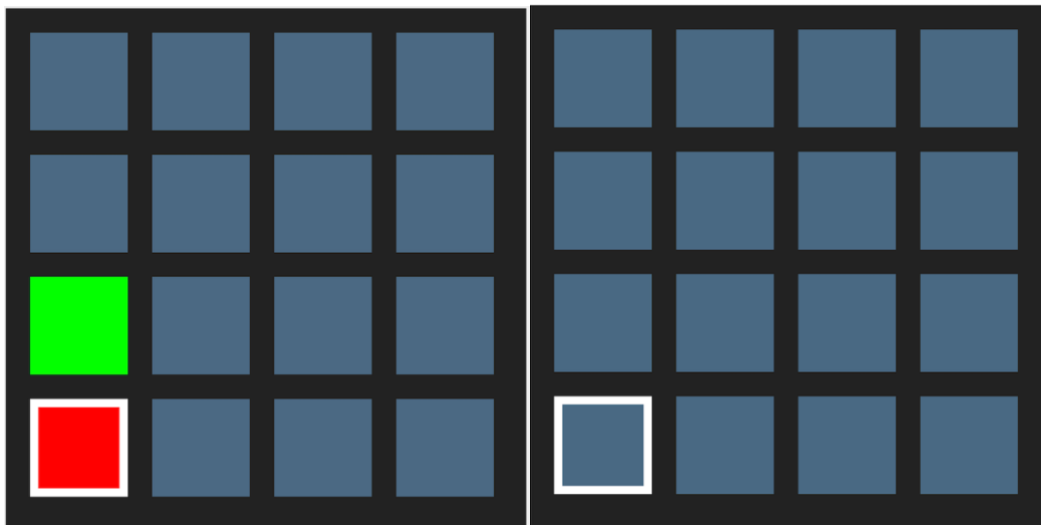
- **Kịch bản 2: Giao diện bàn cờ ban đầu**

- *Mô tả:* Lưới màn hình Bitmap hiển thị 16 ô vuông màu xanh xám, con trỏ ở ô góc trái trên cùng được bao quanh bởi viền màu trắng.



- **Kịch bản 3: Thao tác di chuyển và lật so khớp cặp ô**

- *Mô tả:* Người chơi di chuyển con trỏ, lật hai thẻ khác màu (hai ô hiện màu rồi tự động úp lại sau 800ms) và lật hai thẻ cùng màu (hai ô giữ nguyên trạng thái mở).
- TH1: Khi lật 2 ô khác màu nhau và sau 800ms 2 ô sẽ đóng lại



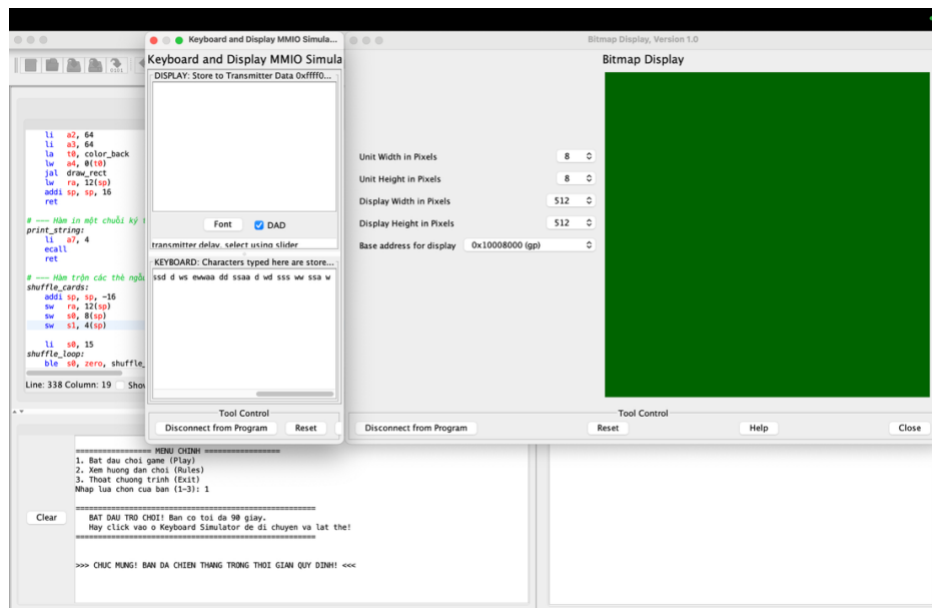
- TH2: Khi lật 2 ô cùng màu, 2 ô giữ nguyên trạng thái mở



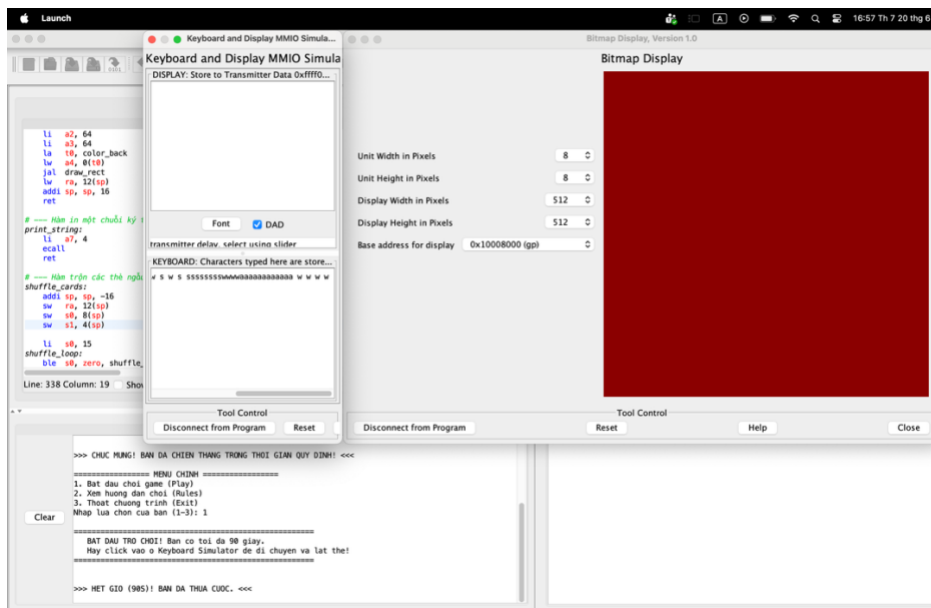
• Kịch bản 4: Trạng thái Kết thúc Trò chơi

- *Mô tả:*

- Trường hợp Thắng: Màn hình Bitmap chuyển hoàn toàn sang màu xanh lá cây đậm và Console xuất dòng chữ chúc mừng.



- Trường hợp Thua (Quá 90s): Màn hình Bitmap chuyển sang màu đỏ đậm và Console thông báo hết giờ.



- Cả hai trường hợp sau 3 giây đều tự động dọn dẹp màn hình và đưa người dùng quay lại Menu chính để chơi lượt tiếp theo nếu muốn.

<<Thắng>>

>>> CHÚC MỪNG! BAN ĐÃ CHIẾN THẮNG TRONG THỜI GIẠN QUY ĐỊNH! <<<

===== MENU CHÍNH =====

1. Bắt đầu chơi game (Play)
 2. Xem hướng dẫn chơi (Rules)
 3. Thoát chương trình (Exit)
- Nhập lựa chọn của bạn (1-3):

<<Thua>>

>>> HẾT GIO (90S)! BAN ĐÃ THUA CƯỢC. <<<

===== MENU CHÍNH =====

1. Bắt đầu chơi game (Play)
 2. Xem hướng dẫn chơi (Rules)
 3. Thoát chương trình (Exit)
- Nhập lựa chọn của bạn (1-3):

V. KẾT LUẬN & HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được của Bài tập lớn

Sau quá trình nghiên cứu tài liệu, thiết kế thuật toán và hiện thực hóa mã nguồn trên công cụ giả lập RARS, Bài tập lớn cuối kỳ đã hoàn thành được các mục tiêu kỹ thuật cốt lõi sau:

- **Về mặt cấu trúc chương trình:** Xây dựng cấu trúc mã nguồn có tính phân cấp rõ ràng, tách biệt các module chức năng thông qua hệ thống chương trình con. Việc tổ chức này giúp mã nguồn dễ đọc, thuận tiện cho việc bảo trì và gỡ lỗi.
- **Về mặt tương tác phần cứng và đồ họa:** Áp dụng thành công cơ chế ánh xạ bộ nhớ MMIO để thu thập dữ liệu từ bàn phím đồng thời với việc giám sát thời gian thực tế của CPU. Bước đầu làm quen và làm chủ thuật toán ánh xạ tọa độ tuyến tính để can thiệp trực tiếp vào bộ đệm khung hình (Framebuffer) của màn hình Bitmap Display.
- **Về mặt trải nghiệm người dùng:** Thiết kế thành công Menu điều hướng có kiểm tra và xử lý ngoại lệ nhập liệu thông thường tại Console Run I/O. Triển khai chức năng khôi phục toàn bộ trạng thái hệ thống (reset_game_state) và xáo trộn thẻ ngẫu nhiên, cho phép người dùng có thể chơi lại nhiều lần liên tục một cách thuận tiện.

2. Hạn chế và Hướng phát triển của đề tài

Mặc dù chương trình đã đáp ứng đầy đủ các yêu cầu từ đề bài, hệ thống vẫn còn tồn tại một số điểm hạn chế cần được cải tiến trong tương lai:

- **Hạn chế của hệ thống hiện tại:**
 - Giao diện hiển thị trên Bitmap Display của RARS còn ở mức cơ bản, chưa hỗ trợ cơ chế hiển thị các ký tự văn bản trực tiếp trên màn hình đồ họa để người dùng dễ dàng theo dõi điểm số hoặc thời gian còn lại.
 - Chương trình chưa tích hợp các hiệu ứng âm thanh tương tác khi lật thẻ đúng hoặc sai để tăng tính sinh động.

- **Hướng phát triển đề tài:**

- **Tích hợp bộ đếm thời gian bằng đồ họa điểm ảnh (Pixel Art):** Nghiên cứu giải pháp thiết kế bộ chữ số dạng đồ họa điểm ảnh để render trực tiếp thời gian đếm ngược lên một phân vùng riêng biệt của màn hình Bitmap.
- **Tích hợp âm thanh kỹ thuật số:** Tìm hiểu và sử dụng thêm dịch vụ hệ thống Syscall 31(Play MIDI tone) hoặc Syscall 33 của giả lập RARS để phát âm thanh thông báo mỗi khi người chơi lật đúng hoặc sai thẻ màu, tạo hiệu ứng phản hồi chân thực hơn.

----- KẾT THÚC -----